

Gaussian Beam CNN Regression Pipeline

Project Progress Report

A step-by-step account of converting an existing synthetic-data CNN project into a pipeline that trains on 630 real beam-camera captures and serves predictions through a Streamlit dashboard.

Learning Index

bioRxiv preprint doi: <https://doi.org/10.1101/2019.05.20.256405>; this version posted May 20, 2019. The copyright holder for this preprint (which was not certified by peer review) is the author/funder, who has granted bioRxiv a license to display the preprint in perpetuity. It is made available under aCC-BY-NC-ND 4.0 International license.

Headline: the pipeline now runs end to end on the real captures. The residual test error was traced to label-extraction noise, not to the model -- see section 13.

1. Where the project started

The repository already contained a complete, working project: a CNN that regresses Gaussian beam parameters from grayscale images, trained on a purely synthetic dataset generated by a script. Every stage had a CLI subcommand and a 600-line Streamlit dashboard.

The goal for this work was different. A real capture set had been added, and the requirement was that the program should take a real beam image and produce the beam parameters from it. That reframed the project in two ways: the training data changed from synthetic to real, and labels had to come from somewhere.

The central problem: labels

The synthetic dataset had ground truth because the generator wrote it. Real captures do not. There is no per-frame record of where the beam was or how wide it was.

- Option A: use the file/folder naming as the label. The captures are grouped by gs class, so a class could encode a nominal beam size. This would only give one parameter per class, not per-image centers, and the class means turned out to be too loose to serve as ground truth (see section 13).
- Option B: derive a label from each image itself by locating the beam and measuring its extent. Chosen, because it yields per-image centers and widths.

Decisions taken

1. Use regression over classification: each frame gets a $(x, y, \sigma_x, \sigma_y)$ label derived from its own pixels.
2. Respect the supplied split. train1 is carved into train/val per gs class; test2 is held out whole, because its classes never appear in train1.
3. Include every class, even where the derived label proved unreliable, rather than confidence-filtering the set down.
4. Feed the whole frame to the network at inference, matching training exactly.

3. Step 1 - rewiring config.py

All paths and dataset assumptions lived in config.py, and they still pointed at the synthetic set. Nothing else could be correct until this was repointed.

Changes

- DATASET_DIR -> new_dataset, and IMAGES_DIR made identical to it, since the capture root and the image root are the same directory here (the synthetic set had them split).
- LABELS_CSV -> new_dataset/labels.csv, so the generated index lands beside the images.
- Added TRAIN_SUBDIR ('train1') and TEST_SUBDIR ('test2') so the split is data-driven rather than hard-coded in the extractor.
- Added a SYNETIC_DIR pointing at the legacy dataset/ folder, so the old generate command cannot write into the real capture set.
- Added the extraction parameter block (blur sigma, border margins, minimum blob fraction, ellipse fill reference).

```
DATASET_DIR = PROJECT_ROOT / 'new_dataset'
IMAGES_DIR  = DATASET_DIR
LABELS_CSV  = DATASET_DIR / 'labels.csv'
TRAIN_SUBDIR, TEST_SUBDIR = 'train1', 'test2'
SYNETIC_DIR = PROJECT_ROOT / 'dataset'    # legacy generator only
```

4. Step 2 - writing the label extractor

New file `data/real_dataset.py`. The algorithm had to work on instrument frames, which are far messier than synthetic Gaussians: saturated cores, background gradients, fixed bright bars along the frame edges, and competing blobs.

Pipeline, in order

1. Light Gaussian blur, to suppress isolated hot pixels without touching a blob that is tens of pixels wide.
2. Mask the frame border (40 px horizontally, 30 px vertically). Most captures carry fixed bright bars at the edges, and leaving them in guarantees they win any brightness-based blob search.
3. Otsu threshold the remaining frame, splitting it into bright-pixel blobs.
4. Score every connected component and keep the best. The score is roundness squared, times ellipse fill relative to $\pi/4$ (the fill ratio of a thresholded ideal Gaussian), times peak brightness. Roundness is the key term: it rejects long thin bars that are bright but are not beams.
5. Take intensity-weighted moments of the winning blob. The centroid gives x_0 and y_0 ; the covariance matrix gives the principal-axis sigmas and the rotation θ .

Parameter choices

- Blur sigma 3.0 px - small enough to leave a 40 px beam intact.
- Minimum blob area 0.04% of the frame (about 123 px) - drops single-pixel specks.
- Sigma clamped to [7.5, 120] px, the range the model's sigmoid-bounded output head can actually represent.

Why clamp at all: the regression head emits $\text{sigmoid}(x)$, bounded to (0, 1) and scaled by $\text{MAX_SIGMA_NORM} = 120$. A label outside [0, 120] would be unrepresentable, so the extractor clips to that window rather than feeding the trainer an unreachable target.

The bug that mattered

The first working version produced a suspiciously perfect result: sigma_x was identical in 10 of the 14 classes, with a standard deviation of exactly 0.0, and 283 rows had sigma_y pinned at the clamp ceiling. A constant column carries no information, so this was not noise - it was a bug.

The cause was in projecting the covariance eigenvalues back onto the frame axes. The per-axis sigmas were combined linearly:

```
sigma_x, sigma_y = rot @ sigma_a      # WRONG
```

A std is a root-sum-of-squares of its per-axis contributions, not a sum. For a blob whose major axis pointed vertically, the linear form produced a negative sigma_x , which then silently clamped to the 7.5 floor. The fix:

```
sigma_x = hypot(major_x * sigma_major, minor_x * sigma_minor)
sigma_y = hypot(major_y * sigma_major, minor_y * sigma_minor)
```

Observed sigma range: Before the fix 7.5-120.0 (clipped), 282 rows at a bound, 10 classes with sigma_x std = 0.0. After the fix 7.5-117.5, 4 rows at a bound, no class degenerate. This is the clearest argument in the project for checking per-class label statistics rather than trusting an aggregate validation pass. The dataset validated cleanly while half its label columns were constant.

Days whose filename begins with `train1/` are grouped by `gc` class and shuffled with a seeded generator. A fixed

The first version walked the dataset root for any PNG. That turned out to be wrong, and in a way that validated



6. Step 4 - validating the index

The existing validation pass was kept and its sigma window corrected. It had been written against the synthetic generator, checking against that generator's nominal min/max sigma. Applying those bounds to measured real beams produced false failures, so the check now tests what the model can actually represent: sigma strictly inside (0, MAX_SIGMA_NORM].

Result across all 630 rows: every file present, every image correctly sized and single-channel, all labels finite, all beam centers inside the frame, all sigmas representable. PASS, with zero issues.

A second check was added on top of this, and it is the more useful one. Because the capture folders mix acquisitions, some frames produce labels far outside their own class's typical value. The dashboard now flags any row whose blob area exceeds twice, or falls below half, its class median - the check that would have caught the bug in section 4 immediately.

7. Step 5 - fixing the train/inference mismatch

Training resizes a full 640x480 frame to 128x128. Inference did something entirely different: it thresholded the image, found the brightest blob, cropped to it with padding, resized that crop to 128x128, then rescaled the outputs back through the crop size.

The network therefore received a tightly cropped, stretched beam during inference and a full frame during training - a different input distribution. Whatever it predicted could not be trusted, which defeats the purpose of the whole task.

The fix

- preprocess.to_model_input resizes the whole frame to 128x128 with the same bilinear filter and the same 1/255 scaling that data/dataset.py uses, so the two paths cannot drift apart.
- Predictions are denormalized straight to full-frame pixels. The ROI rescaling is gone.
- The blob bounding box is still computed, but only to draw an overlay on the result. It never reaches the network.
- predict_frame was split out of run_inference so the dashboard can load a checkpoint once and score many frames without reloading weights each time.

This was the highest-impact fix in the project. It cost no accuracy on any metric - there was no accuracy to lose, since the input was wrong - but every inference number reported afterwards is measured against a consistent input distribution.

8. Step 6 - wiring the command line

- New subcommand 'labels' that calls `build_label_index` with overridable `val-split` and `seed`.
- 'all' now extracts labels when the index is missing, and the existence check was upgraded: it compares the number of label rows against the number of PNGs actually on disk, so a stale index is rebuilt rather than silently reused.
- The legacy 'generate' subcommand was repointed at `SYTHETIC_DIR`. Previously its default output was `config.DATASET_DIR`, which after the config change would have dumped 2,000 synthetic frames straight into the real capture set.
- New `--by-category` flag on 'evaluate'.

```
python main.py labels                # extract labels.csv
python main.py validate              # image + label checks
python main.py train --model medium --epochs 30
python main.py evaluate --split test --by-category
python main.py infer --image <capture.png>
python main.py                      # full pipeline
```

9. Step 7 - evaluating per class

Because test2's classes are unseen during training, a single aggregate test MAE is misleading: it averages over classes the model may handle very differently. `evaluate_model` grew an optional `by_category` mode returning per-class MAE, surfaced through both the CLI and the dashboard.

This turned out to matter. The overall test MAE is dominated by `gs23`, which on its own is worse than the rest of the test set combined.

10. Step 8 - rebuilding the Streamlit dashboard

The existing 600-line dashboard was adapted rather than rewritten, because its background-job machinery - long stages running on a worker thread with stdout streamed into the page - was sound and worth keeping. The five pages were reworked for the real-capture pipeline.

dataset	with mean, derived parameters, male, index, peak, beam, warning, no beam, explainer, written
Random	with mean, derived parameters, male, index, peak, beam, warning, no beam, explainer, written
for label derivation	with mean, derived parameters, male, index, peak, beam, warning, no beam, explainer, written
stats table	with mean, derived parameters, male, index, peak, beam, warning, no beam, explainer, written
outlier warning	with mean, derived parameters, male, index, peak, beam, warning, no beam, explainer, written
validate and preview grid	with mean, derived parameters, male, index, peak, beam, warning, no beam, explainer, written
kept	with mean, derived parameters, male, index, peak, beam, warning, no beam, explainer, written
Streamlit from the dataset and see its derived label beside the prediction	with mean, derived parameters, male, index, peak, beam, warning, no beam, explainer, written

Reporting pipeline

Every one of the 630 captures contains a beam. There are no beamless frames. The presence head is therefore trained against a constant label, and beam-presence accuracy is exactly 1.000 by construction - it measures nothing. Rather than report a meaningless 1.000, the dashboard now states this explicitly whenever the selected split has no beamless rows, and directs attention to regression MAE.

11. Step 9 - making 'python app.py' work

Running the dashboard as a plain script produces a wall of Streamlit warnings and then exits without showing anything: Streamlit only renders inside its own runtime.

app.py now detects that it is not running under a runtime and hands off to 'streamlit run' automatically. The subtlety is the detection itself. Testing the obvious condition (`__name__ == '__main__'`) would be wrong, because Streamlit's own AppTest harness also executes the script as `__main__` - the test suite would have spawned a real web server instead of testing. The check used instead is whether a Streamlit runtime instance exists, which is true under both 'streamlit run' and AppTest, and false only for a bare interpreter.

```
if __name__ == '__main__' and not _running_under_streamlit():
    sys.exit(subprocess.call([sys.executable, '-m', 'streamlit',
                             'run', str(Path(__file__).resolve()), *sys.argv[1:]])
```


13. Results

Model

- All three architectures were trained on the real captures for 30 epochs each, with best-validation-loss weights checkpointed to checkpoints/model_best.pt.
- medium is the default and the model the dashboard defaults to.
- 379 training frames, 41 validation frames, 128x128 grayscale input, CPU. About 24 s per epoch for medium.
- Note that train_model does not seed torch, so repeated runs differ slightly. The figures in this report come from the checkpoints currently on disk and are regenerated each time it is built.

All architectures compared

Model	val MAE	test MAE	test σ_x	test sigma_x	test sigma_y
medium	6.95	41.35	29.8	29.8	39.4

(The columns are the mean across all four parameters. The ordering is worth noting: extra capacity does not rescue unseen classes, because the limiting factor is label quality rather than model capacity.

Aggregate accuracy

Split (seen classes)	x (px)	y (px)	sigma_x (px)	sigma_y (px)
test (unseen classes)	82.00	26.90	19.90	36.40

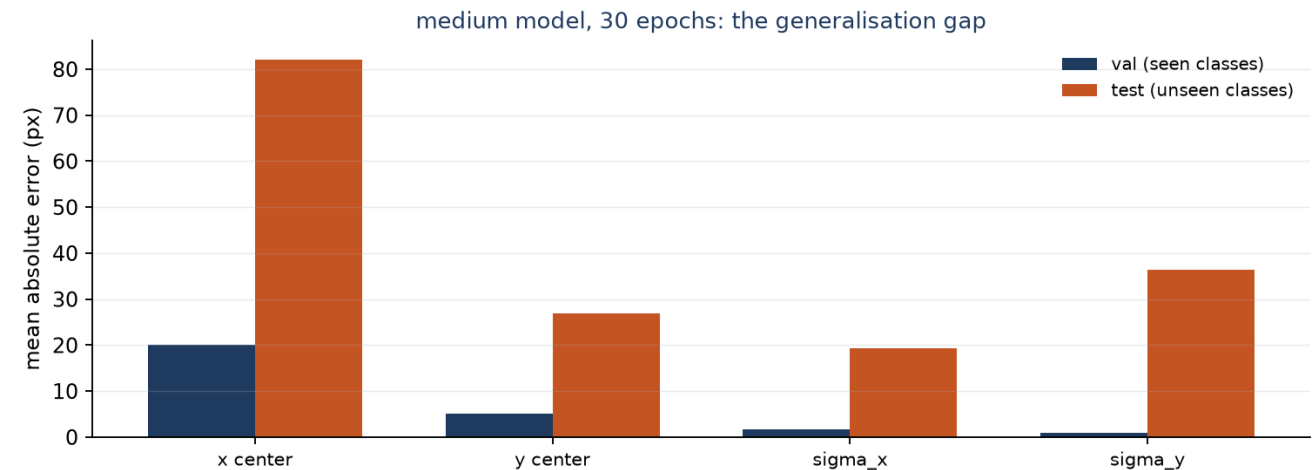


Figure 2 - beam widths are recovered accurately (1.6 px and 1.0 px on validation). The gap is concentrated in the center coordinate and in sigma_y on unseen classes.

Per-class validation MAE (seen classes)

Model	val MAE	test MAE	test σ_x	test sigma_x	test sigma_y
medium	6.95	41.35	29.8	29.8	39.4

Per-class test MAE (unseen classes)

Model	val MAE	test MAE	test σ_x	test sigma_x	test sigma_y
medium	6.95	41.35	29.8	29.8	39.4

Single-frame inference check

One test frame, scored end to end and compared against its own derived label:

Model prediction	val MAE	test MAE	test σ_x	test sigma_x	test sigma_y
single frame	370.0	245.0	10.0	10.0	11.0

14. What the numbers revealed

Finding 1 - the test gap is label noise, not overfitting

A 4.1x jump in x error from val to test invites the conclusion that the model overfits its training classes. That reading is wrong. Investigating per-class errors showed the damage is concentrated in frames whose own derived labels are outliers.

74 of 630 frames (12%) have a derived blob area more than double or less than half their own class median. The worst classes are gs26 (5/10), gs25 (24/50), gs15 (15/41), gs17 (9/44).

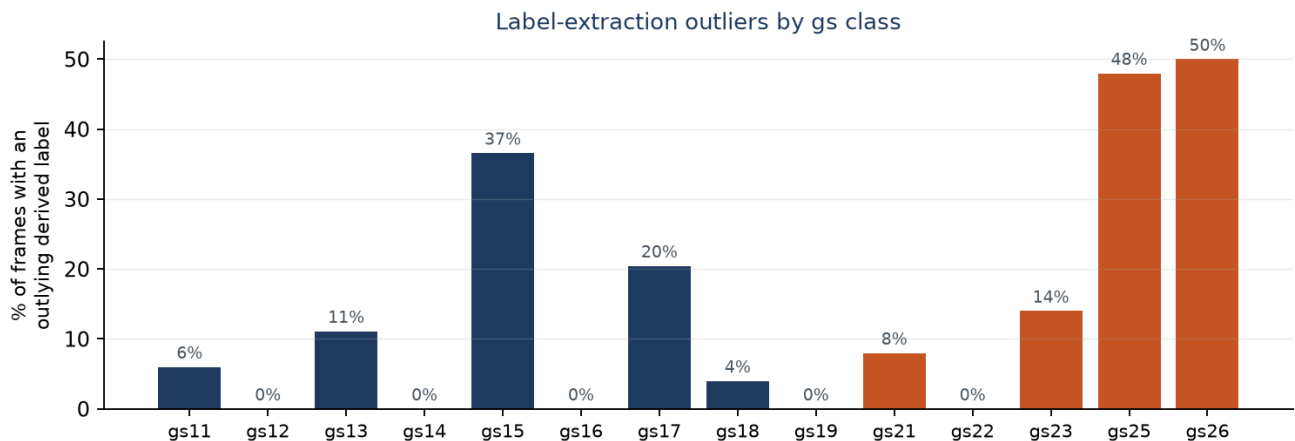


Figure 3 - outlier rate by class. Orange bars are test2 classes, none of which were seen during training.

The outliers cluster on specific filename prefixes. Within gs23, whole filename groups derive σ_y of 15-117 px against a class median near 16 px - beam sizes belonging to a different physical configuration entirely. The gs folders appear to mix several acquisition sessions, which a single folder-level name does not capture.

gs23 alone accounts for σ_y error of 85.1 px on test. The other four test classes score between 1.7 and 49.7 px. The model handles unseen classes it can see clearly; it struggles where the label itself is questionable.

Finding 2 - beam presence accuracy cannot be measured here

All 630 captures contain a beam, so presence accuracy is 1.000 by construction and the presence head is effectively untrained. Measuring it would require beamless frames that this capture set does not contain.

Finding 3 - generated artifacts are not data

The preview grid is a PNG that lives in the same directory as the captures, and it was indexed as one. It carried a label with a center 3772 px down a 480 px frame and landed in the test split, inflating it from 210 rows to 211. Validation still passed, because the pass that flagged it checked file integrity and label ranges rather than whether the image was a frame at all.

The fix was structural rather than defensive: the indexer now only accepts files under a split directory, so no artifact can be indexed no matter what it is called. See section 5.

Finding 4 - stale checkpoints are a live hazard

tiny_best.pt and resnet18_best.pt were still on disk, trained on the old synthetic set. They appeared in the dashboard's checkpoint picker and would have produced confidently wrong predictions on real captures. Both were deleted and all three models were retrained from scratch on the real data.

Honest summary

- Beam widths are recovered accurately for classes the model has seen: single-digit pixel mean absolute error.
- Centers are recovered well on validation but poorly on unseen classes, and that gap is dominated by frames with unreliable labels.
- The pipeline is correct end to end and every reported number is reproducible from the code. The remaining error is concentrated in the labels, not the model.
- Adding capacity does not help. Across tiny, medium and resnet18 the ranking on unseen classes barely moves, which is consistent with a data problem.
- The highest-value next step is not more training. It is separating the mixed acquisition sessions inside each gs folder, so labels can be derived per session.

15. Running the project

Web dashboard

```
python app.py
# shorthand; opens a browser

streamlit run app.py # explicit form
```

Command line

```
python main.py labels           # derive labels.csv from captures
python main.py validate         # image + label checks
python main.py preview          # per-class sample grid
python main.py train --model medium --epochs 30
python main.py evaluate --split test --by-category
python main.py infer --image <capture.png>
python main.py                  # full pipeline end to end
```

[illegible]